Design of a Smart Wearable System for Gold Jewellery Hallmark Recognition and Readability Assessment

Priyanka Lingesan, Riddhi Solanki, Pranathi Kaparthi, Krishnaraj N

Vellore Institute of Technology, Vellore, India

priyanka.lingesan2023@vitstudent.ac.in, riddhi.solanki2023@vitstudent.ac.in, pranathi.k2023@vitstudent.ac.in, krishnaraj.n@vit.ac.in

***Abstract*—*Gold jewellery often carries small engraved, stamped or laser-marked hallmark inscriptions that are hard to read by eye. We design and simulate a smart wearable system that reads such inscriptions and estimates whether its own reading can be trusted. The proposed wrist unit (a Seeed XIAO ESP32S3 Sense board with its camera, a white LED, a button and an OLED) sends one JPEG to a companion computer in a single HTTP POST. The companion runs a Python pipeline of image-quality assessment, localisation, two OCR engines (RapidOCR and Tesseract), readability classification and calibrated rejection with recapture hints, and replies with three lines of text. The system reads characters only; it does not verify gold purity, authenticity or BIS certification. On a synthetic benchmark of 1440 images (240 fictitious items, item-level split), RapidOCR reached exact match 0.510 versus 0.188 for Tesseract, and a learned rejector on OCR-uncertainty features cut the area under the risk–coverage curve from 0.274 (raw confidence) to 0.181. No hardware was built: the wearable camera is an untested proposal, its firmware was only compiled and booted in the Wokwi simulator (which has no camera), and the HTTP receiver was tested with a Python client. A laptop-webcam path and an image-upload path feed the same pipeline.*

Keywords*—*wearable computing, hallmark recognition, readability assessment, OCR, selective classification, ESP32.

I. INTRODUCTION

Hallmarks on gold jewellery are small, low-contrast and easily degraded by wear, glare and blur. OCR engines return a confident string even when it is wrong, so a silent misread costs more than a retake. A wrist-worn camera keeps both hands free and sees held objects at useful scale [1], and finger-worn readers show that feedback about alignment and focus helps users capture text [2]. We therefore design a wrist camera unit paired with a companion computer that reads the inscription, estimates how trustworthy the reading is, and tells the wearer how to retake a poor capture.

We do not verify gold. The assistant reads visible characters only; it makes no statement about purity, composition, authenticity or BIS certification. All data are synthetic, and no hardware was built. Contributions: (1) a reproducible synthetic hallmark-inscription benchmark with per-factor degradation labels and an item-level split; (2) a benchmark of two OCR engines and a preprocessing ablation; (3) a calibrated, separated rejector with recapture guidance, compared against raw confidence; (4) a simple wearable design, one HTTP POST from a camera board to the companion, with a receiver that was tested; and (5) an honest account of what was and was not verified.

II. RELATED WORK

Closest to our task, goldsmith's-mark retrieval in silverware matches learned embeddings [3] but neither transcribes characters nor assesses its own reliability; we found no verified work on OCR of engraved hallmarks. Tesseract [4] and PP-OCR [5] are widely used engines, and scene-text surveys report sensitivity to blur and low resolution [6], [7]. Modern networks are over-confident [8], maximum probability is a weak error detector [9], and scene-text recognisers are miscalibrated [10], so we calibrate a learned score instead of trusting engine confidence. The reject option dates to Chow [11], with risk–coverage formulations in [12], [13] and a survey in [14]. For quality we use cheap gradient, contrast and illumination statistics in the spirit of [15], [16].

III. SYSTEM DESIGN

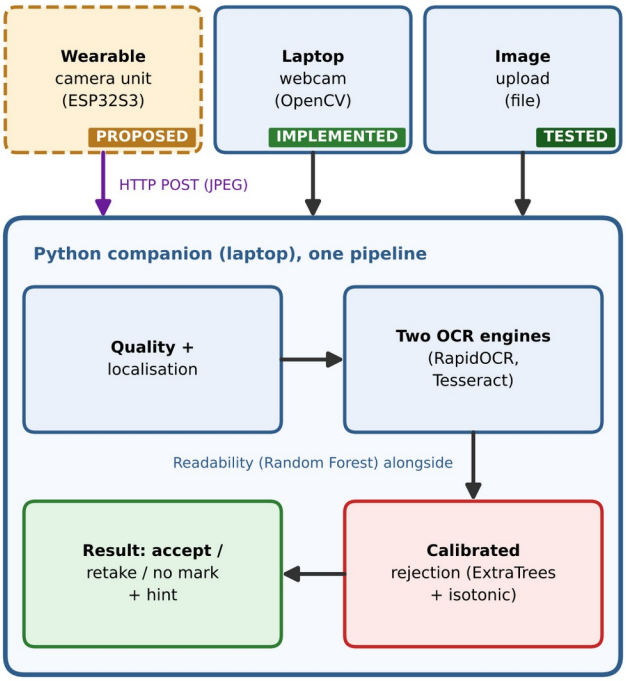


Fig. 1. Architecture. Three sources feed one Python pipeline. The wearable unit is proposed and untested; the webcam path was tested with a test video only.

Wrist unit (proposed, not built). One Seeed XIAO ESP32S3 Sense board (ESP32-S3R8, 8 MB PSRAM) carries a detachable camera module on the board-to-board connector, so the camera needs no external wiring (Fig. 2). The proposed module is the OV5640 with autofocus that the vendor lists for

the Sense; the vendor also states that boards may ship with an OV5640 or an OV3660 (fixed focus) and older pages describe an OV2640, so the fitted module is unconfirmed until a kit is received [17]. The camera is the sensor; the white LED only illuminates the mark. The remaining I/O is an SSD1306 OLED on I²C (D4, D5), a push button on D1 with the internal pull-up, and the LED on D0 through an NPN stage, because a white LED's forward voltage is near the 3.3 V logic level and a GPIO-and-resistor drive would be dim. Power is USB-C 5 V (for example a power bank); battery operation is not designed. Pressing the button lights the LED, captures a JPEG and sends it to the companion; the unit shows the reply on the OLED. It runs no OCR or ML. We chose this board because a DVP camera needs 14 signals, which a DevKit-C-class board with a camera breakout cannot spare, and the AI-Thinker ESP32-CAM leaves only six GPIOs shared with the SD socket and flash LED and has no USB port [18]. No field of view or focus range is listed for these modules, so Fig. 3 uses the OV2640 figures [19] only as an illustrative placeholder (about 23 mm field width at 30 mm), not as OV5640 values; all optical values are estimates and must be measured on the delivered module.

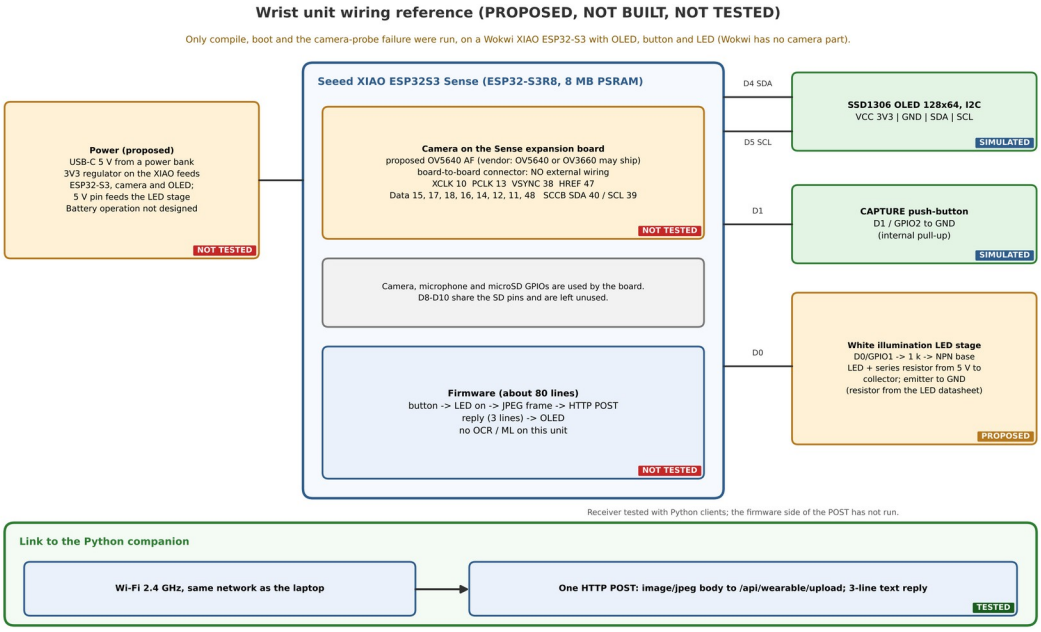


Fig. 2. Proposed wiring reference for the wrist unit. Not built, not powered, not tested. Wokwi has no camera part and mirrors only the button, OLED and LED.

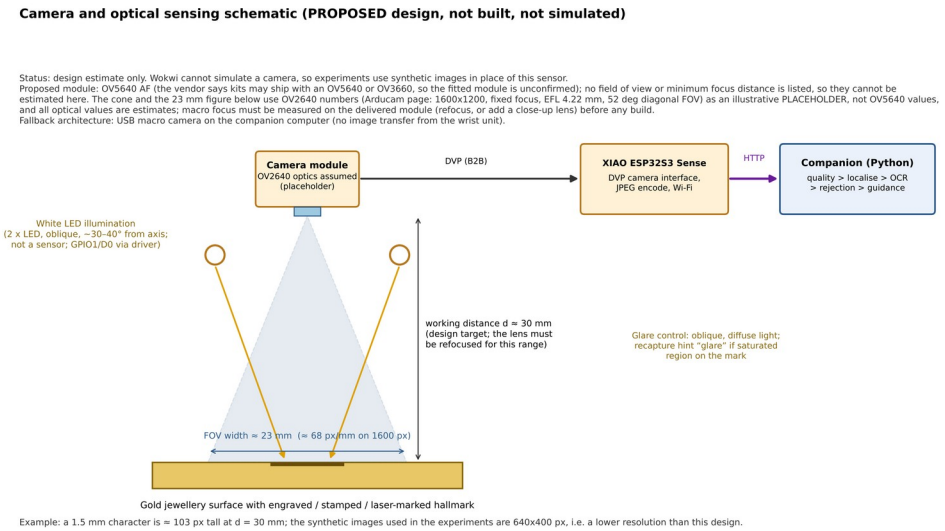


Fig. 3. Camera and optical sensing sketch: an illustrative estimate that uses OV2640 figures [19] as a placeholder (not OV5640 values); not a simulation and not a measurement.

Interface. The wrist unit POSTs the JPEG to /api/wearable/upload on the companion with the content type image/jpeg. The companion stores the bytes unchanged, validates them (decodable, 64 to 8000 pixels per side, not blank), calls the same processing function as the webcam and upload paths, and answers in plain text with three lines: READ OK, RETAKE or NO MARK; the transcription or the recapture hint; and the readability label. A bad image gets a 400 or 422 reply and a server error a 500; no result is ever invented. There is no broker or device state. The link is plain HTTP on a local network without authentication or TLS.

Companion application. Besides the wearable endpoint, the application supports a laptop webcam (OpenCV capture with live preview, retake and disconnect handling) and image upload; all three store the image, read it back and call one function that reduces the longest side to 640 pixels and runs the unchanged analysis pipeline. Identical pixels give identical results through any path; Fig. 4 shows the application.

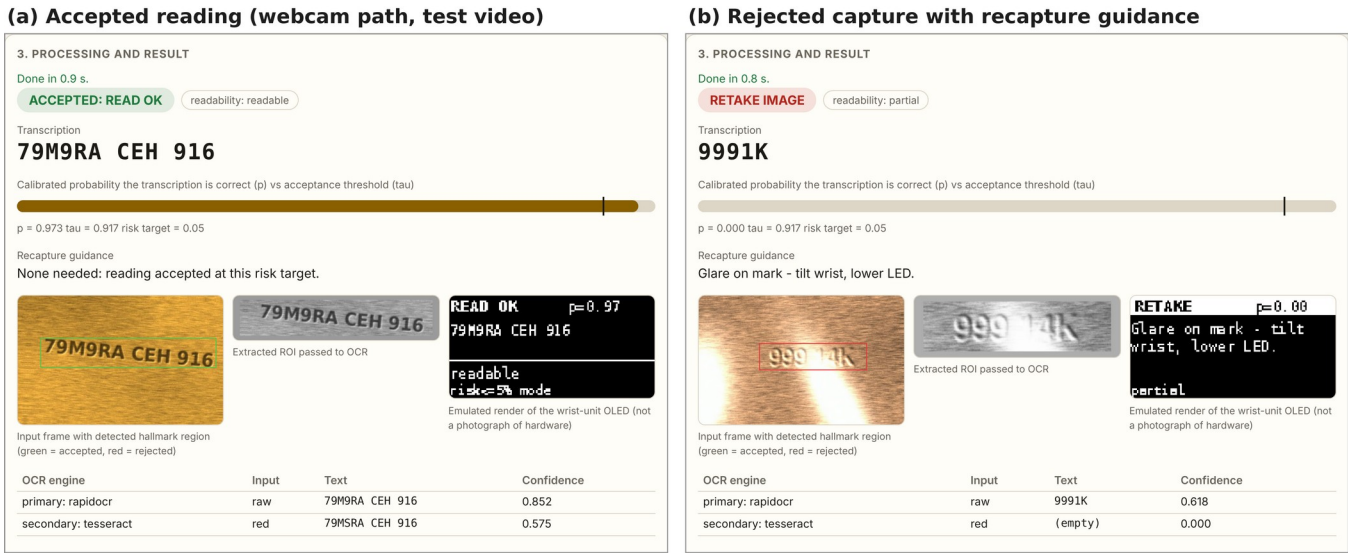


Fig. 4. Result panel of the running companion application (headless-browser test): (a) accepted reading, (b) rejected capture with recapture guidance. The upload path shows the same panel. The camera source is a recorded test video of SYNTHETIC frames; no physical webcam or real jewellery is shown.

IV. RECOGNITION AND REJECTION METHOD

Dataset. The benchmark has 1440 procedurally generated 640×400 images of 240 fictitious hallmark-style inscriptions (fineness numerals such as 375, 585 and 916, karat labels, the string “BIS”, a six-character code and a maker-style mark), with six degraded acquisitions per item, split by item into 144/48/48 items for train/validation/test. They are not photographs of real jewellery and carry no meaning about purity (“BIS” is only a string to be read). Examples are in Fig. 5.



Fig. 5. Examples from the SYNTHETIC dataset (procedural renderings, not photographs of real jewellery).

Pipeline. (1) 21 image-quality features (focus, contrast, illumination, glare, size) are computed; (2) the inscription is localised with a contour method based on the horizontal anisotropy of brushed metal and the region of interest (ROI) is extracted; (3) RapidOCR (PP-OCR recogniser [5]) reads the raw grayscale ROI as the primary reading and Tesseract [4] reads the red channel as the secondary reading; (4) OCR-uncertainty features (confidences, lengths, edit-distance disagreement) are built; (5) a Random Forest on the quality features gives the readability label (readable, partial, unreadable); and (6) a separated rejector, an Extra Trees base model on OCR-uncertainty and quality features with isotonic calibration fitted on out-of-fold scores, gives a probability p that the primary reading is correct. A capture is accepted if $p \geq \tau$, with τ chosen on development data for a target accepted-error rate (empirically, or by a one-sided Clopper–Pearson bound); otherwise the system asks for a retake and gives a rule-based hint from the largest standardised quality deficit (blur, glare, darkness, uneven light, small size).

$$p(x) = C(\, g(\, f(x)\,)\,), \quad accept \;\Leftrightarrow\; p(x) \geq \tau \qquad (1)$$

Metrics. Recognition uses the character error rate CER = Lev(y, ŷ)/|y| and whitespace-insensitive exact match. Selective quality uses the area under the risk–coverage curve (AURC, lower is better), the expected calibration error (ECE) [8] and the paired difference of AURC with a 1000-resample item-level cluster bootstrap. Six rejection strategies are compared:

S1 raw OCR confidence, S2 a hand-set quality gate, S3 a learned score on OCR-uncertainty features, S4 on quality features, S5 on both, and S6 adding grammar validity. Model choice, calibration and thresholds use development items only; the test split (48 items, 288 images) is evaluated once.

V. RESULTS

All experiments ran on a 2-vCPU Intel Xeon at 2.1 GHz (Python 3.13, OpenCV 5.0), not on wearable hardware, and every image is synthetic.

TABLE I. OCR baselines on the test split (48 items, 288 images).

Engine / input	CER	Exact
RapidOCR, raw ROI (primary)	0.271	0.510
Tesseract, red channel	0.579	0.205
Tesseract, raw ROI	0.632	0.188

Recognition. RapidOCR clearly beats Tesseract (exact 0.510, CER 0.271 versus 0.188 for Tesseract on the raw ROI); Table I, Fig. 6). Classical enhancement did not help: the full chain lowered RapidOCR exact match from 0.510 to 0.413, so the final design uses raw crops. This is a negative result for these engines on synthetic data, not evidence that enhancement is useless in general. Accuracy falls with glare, occlusion and combined degradations.

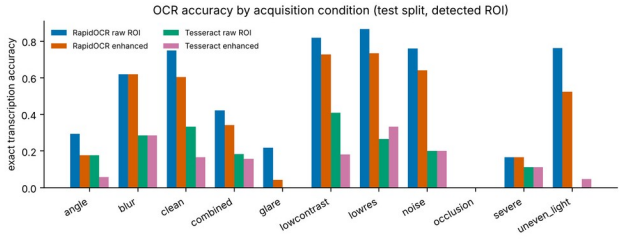


Fig. 6. OCR accuracy by acquisition condition (test, synthetic).

TABLE II. Rejection quality on the test split (S1 raw confidence, S3 OCR-uncertainty, S5 OCR-uncertainty + quality).

strategy	AURC	ECE	AURC 95 % CI
S1	0.274	0.144	[0.168, 0.393]
S3	0.181	0.048	[0.119, 0.264]
S5	0.182	0.042	[0.119, 0.265]

Rejection (Table II, Fig. 7). Raw OCR confidence is a poor reject signal (AURC 0.274, ECE 0.144): its confidence is far above observed accuracy. The learned OCR-uncertainty rejector S3 reaches AURC 0.181 (ECE 0.048), a paired improvement whose interval excludes zero. Adding quality features (S5) did not change AURC measurably (ΔAURC +0.000, CI [-0.009, +0.010]), so on this synthetic data the useful rejection signal is the disagreement and confidence of the two OCR engines; quality features instead drive the recapture hints and the readability proxy (macro-F1 0.804 for quality only; 0.826 with OCR-uncertainty features; confusion matrices in Fig. 8).

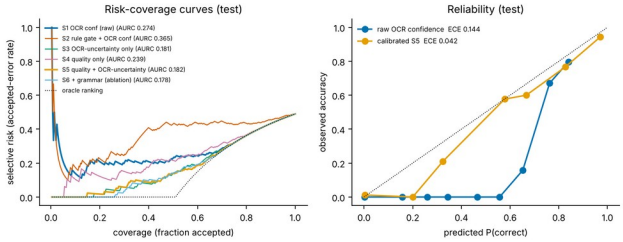


Fig. 7. Risk–coverage curves and reliability diagram (test, synthetic).

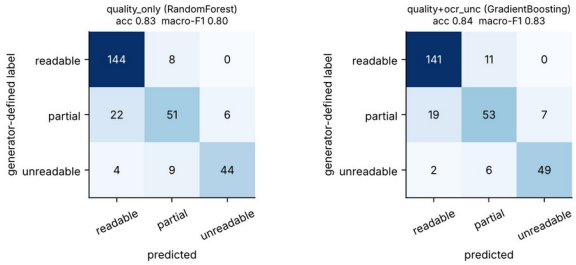


Fig. 8. Readability confusion matrices on the test split (rows: generator-defined proxy label, not human labels). Left: quality features only (Random Forest); right: quality + OCR-uncertainty features. Synthetic data.

TABLE III. Operating points with development-selected thresholds (test split).

strategy	target	tau	cov	err	accepted	n err
S1	0.020	0.877	0.028	0.375	8	3
S1	0.050	0.862	0.090	0.154	26	4
S3	0.020	0.938	0.156	0.000	45	0
S3	0.050	0.861	0.340	0.051	98	5
S5	0.020	0.929	0.250	0.028	72	2
S5	0.050	0.917	0.302	0.057	87	5

Thresholds and guidance (Table III). Development-selected thresholds do not transfer exactly: S5 at a nominal 5 % target accepted 87 captures with 5 errors (5.7 %), and with 48 test items one mistake moves these rates visibly, so we claim no guarantee. Leave-one-source-out testing kept the ordering (the learned rejector beat raw confidence on every held-out rendering source) while OCR exact match varied from 0.454 to 0.590 across sources. Among 201 rejected test captures, 135 had a generator-flagged degradation; the top-1 hint matched it in 0.674 of cases versus 0.185 for a random cause (this tests the hint against the injected cause, not a user's behaviour).

System performance (sandbox CPU, not a Pi). Median end-to-end latency was 264 ms (p95 340 ms) and peak memory 176 MB; Raspberry-Pi-class latency was not measured.

Software and wearable-path checks. 27 automated tests pass. The webcam path was exercised only with a recorded test video of synthetic frames (no physical webcam was available), and the upload path and browser interface with image files. The HTTP receiver was tested with a Python client: the reply equals a direct pipeline call on the stored pixels, the stored file is byte-identical to the sent one, and empty, non-image and truncated bodies are rejected. The firmware (about 80 lines) was loaded in Wokwi on its XIAO ESP32-S3 board with an OLED, a button and an LED: it compiled and booted (Figs. 9 and 10),

and the camera probe failed with ESP_ERR_NOT_SUPPORTED because Wokwi has no camera part [20], after which the firmware printed “camera init failed”. The button, OLED screens, Wi-Fi connection, the POST and any camera frame were not observed, and no frame was ever captured by the wearable camera.

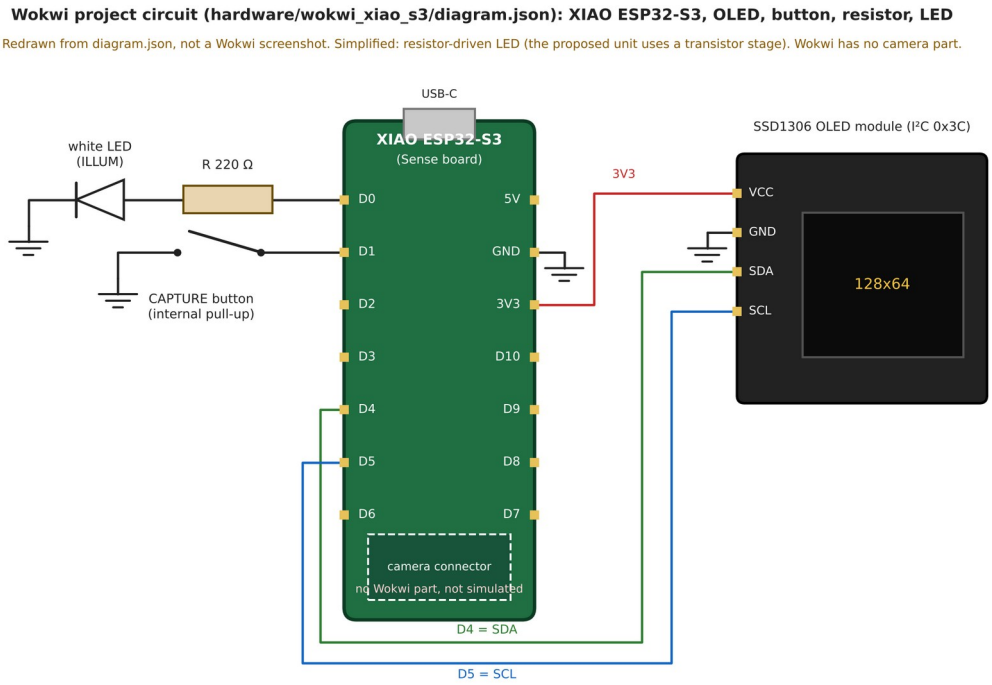


Fig. 9. Circuit of the Wokwi project (hardware/wokwi_xiao_s3/diagram.json): XIAO ESP32-S3, SSD1306 OLED, button, 220 Ω resistor and white LED. This simplified circuit uses a resistor-driven LED; the proposed unit uses the transistor stage of Fig. 2. Redrawn from diagram.json, not a Wokwi screenshot; Wokwi has no camera part, so the camera is not simulated.

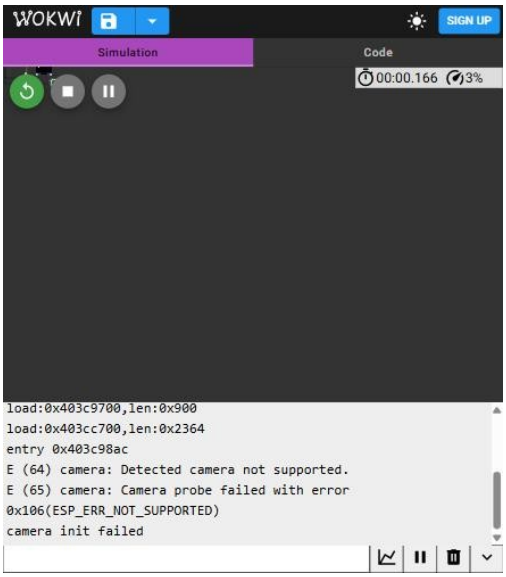


Fig. 10. Serial output of the firmware in Wokwi (XIAO ESP32-S3): boot, then the expected camera-probe failure. This run shows nothing about the camera.

VI. DISCUSSION AND LIMITATIONS

A learned score built on the disagreement and confidence of two OCR engines rejected errors far better than raw confidence, and calibration removed most of the over-confidence. Image-quality features added nothing measurable to rejection once OCR-uncertainty was available; their value here is explanatory, driving the hints. Keeping all recognition on the companion let the wrist firmware stay at about 80 lines and the interface at one request.

Limitations. (1) All data are synthetic and no real gold jewellery was imaged; real metal, lighting and stamping artefacts are not modelled. (2) Readability and per-factor labels come from generator parameters. (3) The test split has only 48 items, so confidence intervals are wide. (4) OCR engines are used off the shelf. (5) The wrist unit, camera and LED stage are a design only; no physical camera was used, the optical figures are first-order estimates, the device-to-companion link was never exercised, and the interface lacks authentication and TLS. (6) There is no user study. (7) Latency and memory come from a different CPU than the intended device. The assistant must not be presented as a way to verify that jewellery is genuine, of a stated purity or BIS-certified.

VII. CONCLUSION

We designed a smart wearable assistant that reads hallmark-style inscriptions and says when its reading should not be trusted: a wrist camera unit sends one JPEG over HTTP to a Python pipeline whose calibrated, OCR-uncertainty-based rejector substantially improves reliability over raw confidence on a synthetic benchmark. The camera unit remains an untested proposal. Next steps are to build the unit and measure its optics and focus, send a real captured frame to the companion, and

evaluate on real, consented hallmark photographs as a separate study; none of the results here should be read as real-world performance.

VIII. REFERENCES

[1] K. Ohnishi, A. Kanehira, A. Kanezaki, T. Harada, "Recognizing Activities of Daily Living with a Wrist-Mounted Camera," Proc. IEEE CVPR, 2016. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2016/html/Ohnishi_Recognizing_Activities_of_CVPR_2016_paper.html

[2] R. Shilkrot, J. Huber, M. E. Wong, P. Maes, S. Nanayakkara, "FingerReader: A Wearable Device to Explore Printed Text on the Go," Proc. ACM CHI 2015, Seoul, 2015. [Online]. Available: http://hdl.handle.net/1721.1/95971

[3] A. Tiwari, V. Christlein, M. Fichtner, F. Gohlke, B. Schübel, T. Witting, H. Zech, M. Zinnen, "Automated Goldsmith's Mark Retrieval in Silverware," arXiv preprint arXiv:2609.20509, 2026. [Online]. Available: https://arxiv.org/abs/2609.20509

[4] R. Smith, "An Overview of the Tesseract OCR Engine," Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), 2007. [Online]. Available: https://research.google/pubs/an-overview-of-the-tesseract-ocr-engine/

[5] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang, H. Wang, "PP-OCR: A Practical Ultra Lightweight OCR System," arXiv preprint arXiv:2009.09941, 2020. [Online]. Available: https://arxiv.org/abs/2009.09941

[6] S. Long, X. He, C. Yao, "Scene Text Detection and Recognition: The Deep Learning Era," Int. J. Comput. Vis., vol. 129 (arXiv:1811.04256), 2021. [Online]. Available: https://arxiv.org/abs/1811.04256

[7] X. Chen, L. Jin, Y. Zhu, C. Luo, T. Wang, "Text Recognition in the Wild: A Survey," arXiv preprint arXiv:2005.03492, 2020. [Online]. Available: https://arxiv.org/abs/2005.03492

[8] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, "On Calibration of Modern Neural Networks," Proc. ICML, PMLR vol. 70, 2017. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

[9] D. Hendrycks, K. Gimpel, "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks," Proc. ICLR, 2017. [Online]. Available: https://arxiv.org/abs/1610.02136

[10] R. Slossberg, O. Anschel, A. Markovitz, R. Litman, A. Aberdam, S. Tsiper, S. Mazor, J. Wu, R. Manmatha, "On Calibration of Scene-Text Recognition Models," Computer Vision - ECCV 2022 Workshops, pp. 263-279, 2022. [Online]. Available: https://doi.org/10.1007/978-3-031-25069-9_18

[11] C. K. Chow, "On optimum recognition error and reject tradeoff," IEEE Trans. Inf. Theory, vol. 16, no. 1, pp. 41-46, 1970. [Online]. Available: https://researchweb.draco.res.ibm.com/publications/on-optimum-recognition-error-and-reject-tradeoff

[12] Y. Geifman, R. El-Yaniv, "Selective Classification for Deep Neural Networks," Advances in Neural Information Processing Systems (NeurIPS) 30, 2017. [Online]. Available: https://arxiv.org/abs/1705.08500

[13] R. El-Yaniv, Y. Wiener, "On the Foundations of Noise-free Selective Classification," J. Mach. Learn. Res., vol. 11, 2010. [Online]. Available: https://jmlr.org/papers/v11/el-yaniv10a.html

[14] K. Hendrickx, L. Perini, D. Van der Plas, W. Meert, J. Davis, "Machine Learning with a Reject Option: A survey," arXiv preprint arXiv:2107.11277, 2021. [Online]. Available: https://arxiv.org/abs/2107.11277

[15] A. Mittal, A. K. Moorthy, A. C. Bovik, "No-Reference Image Quality Assessment in the Spatial Domain," IEEE Trans. Image Process., vol. 21, no. 12, pp. 4695-4708, 2012. [Online]. Available: https://doi.org/10.1109/TIP.2012.2214050

[16] H. Li, F. Zhu, J. Qiu, "Towards Document Image Quality Assessment: A Text Line Based Framework and A Synthetic Text Line Image Dataset," arXiv preprint arXiv:1906.01907, 2019. [Online]. Available: https://arxiv.org/abs/1906.01907

[17] Seeed Studio, "XIAO ESP32S3 (Sense) pin multiplexing and getting-started wiki pages," Seeed Studio wiki (manufacturer documentation), 2026. [Online]. Available: https://wiki.seeedstudio.com/xiao_esp32s3_pin_multiplexing/

[18] JLCPCB, "What Is the ESP32-CAM, and How to Wire, Power, and Program It?," JLCPCB blog (secondary source), 2026. [Online]. Available: https://jlcpcb.com/blog/esp32-cam-guide

[19] Arducam, "Arducam OV2640 Camera Module (2MP Mini CCM, M0031) product page," Arducam (manufacturer page), 2026. [Online]. Available: https://www.arducam.com/product/arducam-ov2640-camera-module-2mp-mini-ccm-compact-camera-modules-compatible-with-arduino_m0031esp32-esp8266-development-board-with-dvp-24-pin-interface_/

[20] Wokwi, "Wokwi supported hardware," Wokwi documentation, 2026. [Online]. Available: https://docs.wokwi.com/getting-started/supported-hardware